SkillNavora · Business AI Stack

COMPLETE GUIDE 08 OF 15

Claude Code

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

BUILD WITH AI · COMPLETE GUIDE

Claude Code: an AI coding agent in your project

Claude Code is an agent that works inside your project folder from the terminal. It reads your code, edits files, runs commands and tests, and uses git, with your permission at each step. This guide sets it up and builds a small feature end to end, with a repeatable workflow you can use for any project.

1. What Claude Code does

- **Reads and edits files** across the project, and searches the code.
- **Runs commands** you approve: tests, builds, linters, git.
- **Plans first** when you ask (plan mode): it proposes steps; you approve before it edits.
- **Remembers project rules** through a file called `CLAUDE.md` in the project root.
- **Extends** with custom commands, skills, hooks and MCP servers (Chapter on MCP).

2. Install and start

1. Install Node.js, then install Claude Code from the official instructions (the command is in its documentation). Sign in with your account when prompted.
2. Open a terminal in your project folder and run `claude`. The first run asks you to trust the folder; only trust folders you know.
3. Run `/init`. It creates a starter `CLAUDE.md`. Edit it to add your stack, test command and style rules.

```
# CLAUDE.md (example)
- Stack: Next.js 15, TypeScript, Supabase. Package manager: npm.
- Run tests with: npm test. Lint with: npm run lint.
- Never edit files in /legacy. Ask before adding a dependency.
- Commit messages: imperative, under 72 characters.
```

3. Worked example: add a feature to a site

Feature: a "Book a counselling call" form on the contact page, validated, saved, and confirmed on screen.

1. **Plan first.** Type: *"Plan a counselling-call form on contact.html: fields, validation, where data is saved, and the tests. Do not edit files yet."* Read the plan; correct it.
2. **Implement.** *"Implement the plan. Keep the existing contact form unchanged."* Approve each file edit as it appears; reject any change you do not understand.
3. **Test.** *"Run the tests. Add tests for an empty phone number and a valid one."* If a test fails, ask it to find the cause before changing code.
4. **Review.** *"Show me the diff and explain each change in plain language."* Look for secrets, debug output and unrelated edits.
5. **Commit.** *"Commit with a clear message."* Push only after your own check in the browser.

4. Permissions and safety

- Choose a permission mode: ask every time (default), accept edits, or plan only. Use the stricter mode for unfamiliar code.
- Keep secrets in `.env` files that are ignored by git; never paste keys into the chat.
- Run it on a branch, not on your main branch, until you trust the workflow.
- Give it a clear finish line: "done when the tests pass and the form works on a phone".

5. Good habits

- One task per session. Use `/clear` between unrelated tasks to keep the context focused.
- Point to files directly (`contact.html`, `functions/api/lead.js`), which is faster than describing them.
- When it gets stuck, ask for the smallest next step rather than a full rewrite.

6. Checklist

- CLAUDE.md written with stack, test command and rules.
- Plan reviewed before any edit.

- Tests pass; diff reviewed; no secrets.
- Commit made on a branch; browser-checked before publishing.

Command names, settings and permission modes are documented in Claude Code's official documentation; check it for your version.